

Лабораторная работа №3

«Стек трейтов»

Скоробогатов С.Ю.

12 марта 2018 г.

1 Цель работы

Целью данной работы является изучение реализации на языке Scala объектно-ориентированного образца проектирования «Decorator» через стек трейтов.

2 Исходные данные

В качестве исходных данных для данной лабораторной работы выступает исходный текст программы, приведённый на листинге 1.

Программа представляет собой заготовку лексического анализатора, спроектированного на основе образца «Decorator».

Класс `Pos` представляет позицию в тексте программы. Его поля `prog`, `offs`, `line` и `col` содержат текст программы, индекс текущей литеры в программе, номер строки и номер колонки, соответственно. Метод `ch` возвращает код текущей литеры или `-1`, если достигнут конец текста программы. Метод `inc` возвращает новую позицию, получаемую путём сдвига по тексту программы вправо на одну литеру. Для создания объекта, представляющего позицию в самом начале текста программы, можно использовать конструктор, принимающий в качестве параметра текст программы:

```
val origin = new Pos("this is program")
```

Объект-синглетон `DomainTags` содержит набор констант типа `Tag`, маркирующих различные лексические домены. В рассматриваемой заготовке лексического анализатора присутствуют только две константы: `WHITESPACE` (участок с пробельными символами) и `END_OF_PROGRAM` (тег псевдолексемы «конец программы»).

Класс `Scanner` является базовым классом для лексических анализаторов. Подразумевается, что его метод `scan`, предназначенный для распознавания лексемы, начинающейся с позиции `start`, будет переопределён в наследниках. Метод `scan` должен возвращать тег лексемы и позицию в тексте программы, непосредственно следующую за лексемой. Реализация метода `scan`, представленная в классе `Scanner` по умолчанию, выдаёт сообщение о синтаксической ошибке в позиции `start`.

Класс `Token` представляет собой итератор по токенам, описывающим последовательность лексем, прочитанную из текста программы. Его поля `tag`, `start` и `follow` содержат тег, позицию начала лексемы и позицию непосредственно после конца лексемы, соответственно. Метод

`image` возвращает строковое представление лексемы. Для получения токена, описывающего следующую лексему, можно использовать метод `next`. Создание объекта `Token` осуществляется путём вызова конструктора, принимающего два параметра: позицию начала лексемы и объект класса `Scanner`, который будет использоваться для распознавания лексем:

```
val firstToken = new Token(new Pos("this is program"), new Scanner)
```

Трейт `Whitespaces` – это декоратор, распознающий непустую последовательность пробельных символов. Он является наследником класса `Scanner` и переопределяет метод `scan` таким образом, чтобы в случае, если в текущей позиции текста программы не находится пробельный символ, вызывалась бы версия метода `scan` следующего класса в линейаризации класса, к которому трейт `Whitespaces` подмешан.

3 Задание

В лабораторной работе предлагается дополнить заготовку лексического анализатора трейтами, осуществляющими распознавание лексем, описанных в таблицах 1–5. Кроме того, нужно реализовать в лексическом анализаторе восстановление при ошибках.

Для проверки работоспособности разработанного лексического анализатора нужно запустить программу вида

```
var t = new Token(  
    new Pos("program source goes here!"),  
    new Scanner  
        with Numbers  
        with Idents  
        with Comments  
        with Whitespaces  
)  
while (t.tag != END_OF_PROGRAM) {  
    println(t.tag.toString + " " + t.start + "-" + t.follow + ": " + t.image)  
    t = t.next  
}
```

Здесь `Numbers`, `Idents`, `Comments` и `Whitespaces` – трейты-декораторы, подмешанные к классу `Scanner` для распознавания числовых литералов, идентификаторов, комментариев и пробелов, соответственно. Конкретный ассортимент декораторов определяется вариантом задания.

Алгоритм 1 Заготовка лексического анализатора

```
1 class Pos private(val prog: String, val offs: Int, val line: Int, val col: Int) {
2   def this(prog: String) = this(prog, 0, 1, 1)
3   def ch = if (offs == prog.length) -1 else prog(offs)
4   def inc = ch match {
5     case '\n' => new Pos(prog, offs+1, line+1, 1)
6     case -1   => this
7     case _    => new Pos(prog, offs+1, line, col+1)
8   }
9   override def toString = "(" + line + ", " + col + ")"
10 }
11
12 object DomainTags extends Enumeration {
13   type Tag = Value
14   val WHITESPACE, END_OF_PROGRAM = Value
15 }
16 import DomainTags._
17
18 class Scanner {
19   def scan(start: Pos): (Tag, Pos) =
20     sys.error("syntax error at " + start)
21 }
22
23 class Token(val start: Pos, scanner: Scanner) {
24   val (tag, follow) = start.ch match {
25     case -1 => (END_OF_PROGRAM, start)
26     case _  => scanner.scan(start)
27   }
28
29   def image = start.prog.substring(start.offs, follow.offs)
30   def next = new Token(follow, scanner)
31 }
32
33 trait Whitespaces extends Scanner {
34   private def missWhitespace(pos: Pos): Pos = pos.ch match {
35     case ' ' => missWhitespace(pos.inc)
36     case '\t' => missWhitespace(pos.inc)
37     case '\n' => missWhitespace(pos.inc)
38     case _    => pos
39   }
40
41   override def scan(start: Pos) = {
42     val follow = missWhitespace(start)
43     if (start != follow) (WHITESPACE, follow)
44     else super.scan(start)
45   }
46 }
```

Таблица 1: Варианты заданий

№	Описание	Студент	Дата
1	Комментарии: начинаются с «(*)» или «{», заканчиваются на «*)» или «}» и могут пересекать границы строк текста. Идентификаторы: последовательности латинских букв, представляющие собой конкатенации двух одинаковых слов («zz», «abab»). Ключевые слова: « ifif », «do», «dodo».	Агазаде	12.03
2	Идентификаторы: начинаются с латинской буквы, за которой в круглых скобках следует непустая последовательность десятичных цифр (например, «i(1)», «A(50)»). Числовые литералы: последовательности десятичных цифр, в которых группы по три разряда разделены точками (например, «10.000.000», «999»). Комментарии: последовательности символов, не содержащие «—», заключённые в «*—» и «—*». Ключевые слова: «Y(0)», «IF(1)», «DO(1)».	Бородин	12.03
3	Идентификаторы: начинаются с латинской буквы «s», «t» или «v», за которой через точку следует непустая последовательность букв и цифр (например, «s.1», «t.al»). Комментарии: строка программы целиком считается комментарием, если в первой позиции в ней стоит «*». Числовые константы: непустые последовательности десятичных цифр. Строковые константы: произвольные последовательности символов, заключённые в апострофы (если строковая константа должна содержать апостроф, он удваивается). Ключевые слова: «\$ENTRY», «\$EXTERN».	Булхак	12.03
4	Комментарии: любые последовательности символов, не содержащие скобок «()» и «{ }». Идентификаторы: любые последовательности символов, заключённые в «{ }». Числовые константы: непустые последовательности десятичных цифр, заключённые в «()». Ключевые слова: «{ if }» и «(while)».	Джаин	12.03
5	Идентификаторы: последовательности латинских букв, начинающиеся со знака подчёркивания. Числа: непустые последовательности латинских букв. Ключевые слова: «_if_» и «while». Комментарии: «/* ... */» (как в языке C).	Зухбая	12.03
6	Идентификаторы: палиндромы, составленные из латинских букв и знаков «минус». Комментарии: от знака «—» до перевода строки или конца файла. Числовые константы: последовательности десятичных цифр, перед которыми может стоять знак «минус». Ключевые слова: «dod», «if».	Климов	12.03

Таблица 2: Варианты заданий

№	Описание	Студент	Дата
7	Идентификаторы: последовательности строго возрастающих десятичных цифр. Строки: последовательности символов, заключённые в «/ /» и не содержащие перевода строки и знака «/». Числовые константы: последовательности десятичных цифр, начинающиеся с точки или не являющиеся строго возрастающими. Ключевые слова: «. if» и «.while».	Ковальский	12.03
8	Числовые литералы – римские (составлены из заглавных букв). Идентификаторы – произвольные последовательности латинских букв. Строковые литералы: начинаются и заканчиваются точкой, не могут содержать перевод строки, для включения точки в состав литерала необходимо поставить две точки подряд. Ключевое слово: «ave.caesar».	Ковега	12.03
9	Идентификаторы: последовательности латинских букв и знаков «минус», начинающиеся с заглавной буквы. Числовые константы: последовательности десятичных цифр, которые могут содержать десятичную точку и перед которыми может стоять знак «минус». Ключевые слова: «if –1», «if –0».	Ковинько	12.03
10	Идентификаторы: последовательности заглавных латинских букв, десятичных цифр и знаков подчёркивания, не состоящие только из цифр. Строковые литералы: либо последовательности строчных латинских букв, либо заключённые в фигурные скобки произвольные последовательности символов, не содержащие перевода строки и фигурных скобок. Числовые константы: последовательности десятичных цифр, перед которыми может стоять знак минус. Ключевые слова: «FOR», «DO».	Лапатин	12.03
11		Максимов	12.03
12	Строковые литералы: ограничены двойными кавычками, могут содержать Escape-последовательности «\»», «\n» и «\t», не пересекают границы строк текста. Числовые литералы: последовательности десятичных цифр, разбитые точками на группы по три цифры («100», «1.000», «1.000.000»). Идентификаторы: последовательности буквенных символов Unicode, знаков подчёркивания и цифр, начинающиеся с буквы или подчёркивания.	Савельев	12.03

Таблица 3: Варианты заданий

№	Описание	Студент	Дата
13	Комментарии: начинаются с «//» и продолжаются до окончания строки текста. Идентификаторы: любой текст, не содержащий «/» и ограниченный символами «/». Ключевые слова: «/while/», «/do/», «/end/».	Терешкина	12.03
14	Идентификаторы: последовательности латинских букв и десятичных цифр, оканчивающиеся на цифру. Числовые литералы: последовательности десятичных цифр, органические знаками «<» и «>». Операции: «<=», «=», «==».	Трифиленкова	12.03
15	Строковые литералы: ограничены апострофами, для включения апострофа в литерал он удваивается, не пересекают границы строк текста. Числовые литералы: последовательности десятичных цифр, которые могут включать точку и предваряться знаком «минус». Идентификаторы: последовательности буквенных символов Unicode, точек и цифр, начинающиеся с буквы.	Тришин	12.03
16	Идентификаторы: последовательности заглавных латинских букв, за которыми могут располагаться последовательности знаков «+», «-» и «*». Числовые литералы: знак «*» или последовательности, состоящие целиком либо из знаков «+», либо из знаков «-». Ключевые слова: «ON», «OFF», «**».	Цуканов	12.03
17	Идентификаторы: произвольные последовательности латинских букв и десятичных цифр, в которых есть хотя бы одна буква. Числовые константы: непустые последовательности десятичных цифр. Строковые литералы: произвольные последовательности символов, начинающиеся со знака «%» и продолжающиеся до конца строки программы. Ключевые слова: «while», «unless», «for».	Антипов	12.03
18	Идентификаторы: последовательности буквенных символов Unicode и десятичных цифр, начинающиеся и заканчивающиеся на одну и ту же букву. Числовые литералы: последовательности шестнадцатеричных цифр (чтобы литерал не был похож на идентификатор, его можно предварять нулём). Ключевые слова: «req», «xx», «xxx».	Атымханова	12.03
19	Символьные литералы: ограничены апострофами, могут содержать Escape-последовательности «\», «\n» и «\xxxx» (в последней Escape-последовательности буквы «x» обозначают шестнадцатеричные цифры). Идентификаторы: последовательности символов Unicode длиной от 2 до 10 символов, начинающиеся и заканчивающиеся буквой. Ключевые слова: «z», «for», «forward».	Барсук	12.03

Таблица 4: Варианты заданий

№	Описание	Студент	Дата
20	Идентификаторы: латинская буква, за которой следует десятичная цифра. Числовые литералы: последовательности шестнадцатеричных цифр. Строковые литералы: произвольные последовательности символов, не включающие перевод строки, обрамлённые знаками «/». Чтобы включить «/» в строковый литерал, его надо удвоить. Ключевые слова: «Q0», «QQQ».	Васина	12.03
21	Идентификаторы: последовательности латинских букв и десятичных цифр, начинающиеся с буквы, после которых через знак подчёркивания может следовать последовательность из символов «+», «-», «*», «/» и «!» (например, «alpha1», «unary_!»). Числовые константы: непустые последовательности десятичных цифр или одиночные символы ASCII, заключённые в апострофы. Комментарии: как в языке C++. Ключевые слова: «class», «if», «for».	Жолтиков	12.03
22	Идентификаторы: последовательности латинских букв, начинающиеся с гласной буквы. Числовые литералы: последовательности десятичных цифр, перед которыми может стоять знак «минус». Операции: «--», «<», «<=».	Калинина	12.03
23	Комментарии: начинаются с «/*», заканчиваются на «*/» и могут пересекать границы строк текста. Идентификаторы: последовательности латинских букв и десятичных цифр, в которых буквы и цифры чередуются. Ключевые слова: «for», «if», «m1».	Карькин	12.03
24	Идентификаторы: либо последовательности латинских букв, либо непустые последовательности десятичных цифр, ограниченные круглыми скобками. Числовые литералы: либо последовательности десятичных цифр, не начинающиеся с нуля, либо «0». Операции: «()», «:», «:=».	Никифоров	12.03
25	Строковые литералы: органичены двойными кавычками, для включения двойной кавычки она удваивается, для продолжения литерала на следующей строчке текста в конце текущей строчки ставится знак «\». Числовые литералы: либо последовательности десятичных цифр, либо последовательности шестнадцатеричных цифр, начинающиеся с «\$». Идентификаторы: последовательности буквенных символов Unicode, цифр и знаков «\$», начинающиеся с буквы.	Овчинникова	12.03
26	Комментарии: целиком строка текста, начинающаяся с «*». Идентификаторы: либо последовательности латинских букв нечётной длины, либо последовательности символов «*». Ключевые слова: «with», «end», «***».	Олохтонов	12.03

Таблица 5: Варианты заданий

№	Описание	Студент	Дата
27	Идентификаторы: произвольные последовательности латинских букв и десятичных цифр, начинающиеся со знака подчёркивания. Числовые литералы: произвольные последовательности десятичных цифр и знаков подчёркивания. Ключевые слова: «_if_», «_». Булевские литералы: 0 и 1 (то есть числа 0 и 1 надо записывать как 00 и 01).	Лукманова	12.03
28	Идентификаторы: непустая последовательность латинских букв, за которой следует непустая последовательность десятичных цифр. Числовые константы: последовательности десятичных цифр, которые могут включать десятичную точку. Ключевые слова: «if1», «then». Строковые литералы: произвольные последовательности символов, обрамлённые точками.	Рудикова	12.03
29	Идентификаторы: последовательности латинских букв и десятичных цифр, заканчивающиеся буквой. Числовые константы: последовательности десятичных цифр и знаков подчёркивания, не начинающиеся и не заканчивающиеся подчёркиванием. Строковые литералы: произвольные последовательности символов, обрамлённые знаками подчёркивания (чтобы включить знак подчёркивания в строковый литерал, его надо удвоить). Ключевые слова: «if», «_».	Соколов	12.03
30	Строковые литералы: ограничены обратными кавычками, могут занимать несколько строчек текста, для включения обратной кавычки она удваивается. Числовые литералы: десятичные литералы представляют собой последовательности десятичных цифр, двоичные – последовательности нулей и единиц, оканчивающиеся буквой «b». Идентификаторы: последовательности десятичных цифр и знаков «?», «*» и « », не начинающиеся с цифры.	Цалкович	12.03